

myAssistente5

Assistente Personale Desktop – Toolkit Documentazione

(7) Manuale AIML avanzato · Release 5 · italiano

[Home](#)[Autore](#)[GB English](#)[IT Italiano](#)

Contenuto: [Test coniglio](#) [Personalità avatar](#) [Documentazione](#)

(7) Manuale AIML avanzato

- **Scopo:** Questo documento spiega come funziona il test del “coniglio” usando il linguaggio **AIML** (Artificial Intelligence Markup Language).
- **Prerequisiti:** myAssistente5 installato (vedi (1) *Manuale installazione*) e File AIML testConiglio.aiml.
- **Riferimenti:** (6) *Manuale AIML*.

In breve: myAssistente5 riconosce quando scrivi una frase con la parola “**coniglio**” e risponde in modo sempre più “coinvolto”, come se stesse seguendo una storia che si sviluppa passo dopo passo.

1. Il trigger: quando myAssistente “sente” la parola CONIGLIO

La prima cosa che fa il file AIML è dire:

- “Quando l’utente scrive una frase che contiene la parola **CONIGLIO**, manda tutto a un **gestore centrale**”.

```
<category>
  <!-- Cattura qualsiasi frase con la parola CONIGLIO -->
  <pattern>* CONIGLIO *</pattern>
  <template>
    <!-- Reindirizza al gestore principale -->
    <srai>CONIGLIO.GESTORE</srai>
  </template>
</category>
```

Tradotto in linguaggio umano:

- **Pattern** * CONIGLIO * significa: "qualsiasi frase che contenga la parola CONIGLIO".
- **SRAI** è un modo per dire: "non rispondere subito, ma passa il controllo a un'altra regola chiamata CONIGLIO.GESTORE".

2. Il gestore centrale: decide "a che punto della storia" siamo arrivati

Il gestore centrale fa due cose:

1. Controlla a che **step** della storia del coniglio sei.
2. Ti manda alla risposta giusta per quello step.

```
<category>
  <pattern>CONIGLIO.GESTORE</pattern>
  <template>
    <!-- Se la memoria è vuota, inizializza a 1 -->
    <think>
      <set name="coniglio_step">
```

```
        <condition name="coniglio_step">
            <li value="">1</li>
            <li> <get name="coniglio_step"/> </li>
        </condition>
    </set>
</think>

    <!-- Ora reindirizza allo step corretto -->
    <srai>CONIGLIO.STEP.<get name="coniglio_step"/></srai>
</template>
</category>
```

Qui compaiono due concetti importanti:

- `<set>` e `<get>`: sono usati per **memorizzare** e **recuperare** un valore, come una variabile. In questo caso, la variabile si chiama **coniglio_step**.
- `<condition>`: serve per dire “se la variabile è vuota, mettila a 1, altrimenti lascia il valore che ha già”.

In pratica:

- Se è la **prima volta** che parli di conigli, lo step diventa **1**.
- Se ne hai già parlato, lo step rimane quello che era.
- Alla fine ti manda alla regola CONIGLIO.STEP.X, dove X è il numero dello step.

3. Gli step: una storia che si sviluppa

Ogni **step** è un “pezzo di storia” che myAssistente ti racconta sul coniglio. Ogni volta che nomini di nuovo “coniglio”, my Assistente:

1. Guarda a che step sei.
2. Ti dà la risposta corrispondente.

3. Aumenta lo step di 1, così la prossima volta la storia va avanti.

3.1 Esempio: Step 1

```
<category>
  <pattern>CONIGLIO.STEP.1</pattern>
  <template avatar="default">
    Quale coniglio?
    <think><set name="coniglio_step">2</set></think>
  </template>
</category>
```

Che cosa succede qui?

- myAssistente ti risponde: **“Quale coniglio?”**
- In background (con <think>) aggiorna la memoria: **coniglio_step = 2.**

La prossima volta che scriverai una frase con “coniglio”, il gestore centrale ti manderà allo **STEP 2**.

3.2 Esempio: Step 2

```
<category>
  <pattern>CONIGLIO.STEP.2</pattern>
  <template avatar="default">
    Sei sicuro di aver visto un coniglio?
    <think><set name="coniglio_step">3</set></think>
  </template>
</category>
```

Qui myAssistente diventa un po' più sospettoso:

- Risponde: **“Sei sicuro di aver visto un coniglio?”**
- Porta lo step a **3**.

E così via, fino allo **STEP 25**, dove il tono diventa quasi “meta-narrativo”:

```
<category>
  <pattern>CONIGLIO.STEP.25</pattern>
  <template avatar="default">
    Se continui a parlarne, potrei iniziare a sospettare che il coniglio stia
    <think><set name="coniglio_step">26</set></think>
  </template>
</category>
```



4. Ramificazioni: quando l'assistente reagisce alle tue risposte

Oltre alla “linea principale” della storia, myAssistente può reagire in modo diverso a seconda di come rispondi in certi punti chiave.

Per esempio, dopo una domanda come:

Dimmi la verità: ti piacciono i conigli?

myAssistente può distinguere tra “SI” e “NO” usando un altro elemento di AIML: **<that>**.

4.1 Esempio: risposta “SI”

```
<category>
  <pattern>SI</pattern>
  <that>Dimmi la verità: ti piacciono i conigli?</that>
  <template avatar="default">
    Lo immaginavo. Allora la tua percezione potrebbe essere influenzata.
  </template>
</category>
```

Qui l'assistente controlla due cose:

- Hai scritto **“SI”**.
- La sua frase precedente era esattamente: **“Dimmi la verità: ti piacciono i conigli?”**.

Solo se entrambe le condizioni sono vere, myAssistente risponde in quel modo.

4.2 Esempio: risposta “NO”

```
<category>
  <pattern>NO</pattern>
  <that>Dimmi la verità: ti piacciono i conigli?</that>
  <template avatar="default">
    Interessante. Allora perché ti ha colpito così tanto?
  </template>
</category>
```

In questo modo, l'assistente non solo segue una storia lineare, ma può anche **ramificare** la conversazione in base alle tue risposte.

5. Reset: ricominciare la storia da capo

Se vuoi azzerare tutto e ripartire da zero, il file AIML prevede un comando speciale:

```
<category>
  <pattern>RESET CONIGLIO</pattern>
  <template avatar="default">
    Memoria del coniglio azzerata.
    <think><set name="coniglio_step">1</set></think>
  </template>
</category>
```

Quando scrivi **RESET CONIGLIO**:

- myAssistente conferma che ha azzerato la memoria.
- Lo step torna a **1**, come se fosse la prima volta che parli di conigli.

Che cosa ti porti a casa dal test del coniglio

Anche senza conoscere AIML in dettaglio, puoi intuire che myAssistente:

- Riconosce quando parli di **conigli**.
- Ricorda **a che punto** della storia siete arrivati.
- Fa evolvere la conversazione passo dopo passo, fino a 25 livelli.
- Può reagire in modo diverso a seconda delle tue risposte ("SI", "NO", ecc.).
- Può essere **resettato** per ricominciare da capo.

Tutto questo è ottenuto con:

- **Pattern** per riconoscere cosa scrivi.
- **Template** per decidere cosa risponde l'assistente.
- **Memoria** (<set> / <get>) per ricordare lo step.
- **Reindirizzamenti** (<srail>) per tenere il codice ordinato.

- **Contesto** (<that>) per capire a quale domanda stai rispondendo.
-

Come sviluppare la storia

A partire da questa struttura, si possono aggiungere:

- altri temi oltre al coniglio (altri “filoni narrativi”);
- più memoria (colore del coniglio, luogo, orario, ecc.);
- personalità diverse di myAssistente (più ironico, più scientifico, più drammatico);
- scelte multiple che portano a finali diversi.

Già così, questo file AIML è un buon esempio di:

- conversazione a più step,
- memoria simulata,
- ramificazioni semplici.

Consideralo un buon punto di partenza per capire come un “semplice” linguaggio a regole possa creare esperienze conversazionali sorprendentemente ricche.

Personalità configurabili

Vedi esempio nel File AIML aggiuntivo testConiglioAVANZATO.aiml

Oltre alla “storia del coniglio”, myAssistente può anche cambiare **personalità**, cioè il suo stile di risposta. Questo è gestito da un altro file AIML che introduce 6 personalità diverse:

- **NEUTRA** – tono standard, equilibrato
- **IRONICA** – tono sarcastico, pungente

- **DRAMMATICA** – tono teatrale, esagerato
- **SCIENTIFICA** – tono analitico, “freddo”
- **PARANOICA** – tono sospettoso, inquieto
- **ZEN** – tono calmo, filosofico

L'utente può scegliere la personalità scrivendo frasi come:

- PERSONALITA NEUTRA
- PERSONALITA IRONICA
- PERSONALITA DRAMMATICA
- PERSONALITA SCIENTIFICA
- PERSONALITA PARANOICA
- PERSONALITA ZEN

Quando lo fa, l'assistente memorizza la scelta in una variabile interna chiamata `persona` usando i tag AIML `<set>` e `<get>`.

```
<category>
  <pattern>PERSONALITA IRONICA</pattern>
  <template avatar="default">
    Personalità impostata su IRONICA.
    <think><set name="persona">IRONICA</set></think>
  </template>
</category>
```

Il “dispatcher”: come myAssistente sceglie la versione giusta della risposta

Per far sì che ogni risposta di myAssistente rispetti la personalità scelta, il file AIML usa un **dispatcher**, cioè una regola centrale che decide *quale variante* della risposta

usare.

L'idea è questa:

1. La logica dell'assistente chiede una certa risposta, per esempio

CONIGLIO_STEP_1 .

2. Invece di rispondere subito, passa dal dispatcher: PERSONA.DISPATCH

CONIGLIO_STEP_1 .

3. Il dispatcher guarda la variabile `persona` e reindirizza alla versione corretta:

- CONIGLIO_STEP_1.NEUTRA
- CONIGLIO_STEP_1.IRONICA
- CONIGLIO_STEP_1.DRAMMATICA
- CONIGLIO_STEP_1.SCIENTIFICA
- CONIGLIO_STEP_1.PARANOICA
- CONIGLIO_STEP_1.ZEN

```
<category>
  <pattern>PERSONA.DISPATCH *</pattern>
  <template>
    <condition name="persona">
      <li value="IRONICA"><srai><star/>.IRONICA</srai></li>
      <li value="DRAMMATICA"><srai><star/>.DRAMMATICA</srai></li>
      <li value="SCIENTIFICA"><srai><star/>.SCIENTIFICA</srai></li>
      <li value="PARANOICA"><srai><star/>.PARANOICA</srai></li>
      <li value="ZEN"><srai><star/>.ZEN</srai></li>
      <!-- Default: NEUTRA -->
      <li><srai><star/>.NEUTRA</srai></li>
    </condition>
  </template>
</category>
```

In pratica:

- `<star/>` rappresenta il nome della risposta richiesta (es. `CONIGLIO_STEP_1`).
 - Il dispatcher aggiunge il suffisso della personalità (es. `.IRONICA`) e chiama la categoria corrispondente.
-

La stessa risposta, 6 stili diversi

Ogni “pezzo di storia” sul coniglio può avere 6 varianti, una per personalità. Per esempio, lo **STEP 1** (la prima reazione al coniglio) può essere:

```
<!-- NEUTRA -->
<category>
  <pattern>CONIGLIO_STEP_1.NEUTRA</pattern>
  <template avatar="default">Quale coniglio?</template>
</category>

<!-- IRONICA -->
<category>
  <pattern>CONIGLIO_STEP_1.IRONICA</pattern>
  <template avatar="default">Un coniglio? Certo, e io sono Babbo Natale.</templ
</category>

<!-- DRAMMATICA -->
<category>
  <pattern>CONIGLIO_STEP_1.DRAMMATICA</pattern>
  <template avatar="default">Un coniglio... il destino ha parlato.</template>
</category>

<!-- SCIENTIFICA -->
<category>
  <pattern>CONIGLIO_STEP_1.SCIENTIFICA</pattern>
  <template avatar="default">Definisci “coniglio”: Leporidae, ordine Lagomorpha
```

```
</category>

<!-- PARANOICA -->
<category>
  <pattern>CONIGLIO_STEP_1.PARANOICA</pattern>
  <template avatar="default">Un coniglio? Sei sicuro che non ti stia seguendo?</template>
</category>

<!-- ZEN -->
<category>
  <pattern>CONIGLIO_STEP_1.ZEN</pattern>
  <template avatar="default">Il coniglio è solo un pensiero che attraversa la mente</template>
</category>
```

La “logica” della storia (step 1, step 2, step 3, ecc.) può rimanere la stessa, ma il **tono** cambia completamente in base alla personalità.

Reset della personalità

Per riportare myAssistente alla personalità predefinita (NEUTRA), il file AIML prevede un comando:

```
<category>
  <pattern>RESET PERSONALITA</pattern>
  <template avatar="default">
    Personalità riportata allo stato predefinito (NEUTRA).
    <think><set name="persona">NEUTRA</set></think>
  </template>
</category>
```

Dopo questo comando, le risposte torneranno a usare le varianti **.NEUTRA**.

Che cosa cambia per l'utente finale

Dal punto di vista di chi usa l'assistente:

- La storia del coniglio continua a funzionare come prima (step, memoria, ecc.).
- In più, l'utente può "colorare" la storia scegliendo lo stile di myAssistente.
- Lo stesso contenuto può risultare:
 - più serio,
 - più ironico,
 - più inquietante,
 - più filosofico,
 - più "scientifico".

Tecnicamente, tutto questo è ottenuto combinando:

- **Memoria** (<set> / <get>) per ricordare la personalità scelta.
- **Dispatcher** (PERSONA.DISPATCH) per scegliere la variante giusta.
- **Pattern dedicati** per ogni combinazione "step + personalità".

In questo modo, AIML non è solo un elenco di domande e risposte, ma diventa una piccola "macchina narrativa" configurabile.

Documentazione

La documentazione disponibile per **myAssistente5**.

I documenti si trovano anche nella cartella [/DOCS](#) del repository. Per eventuale documentazione mancante, fare riferimento alla documentazione in inglese o in italiano delle release precedenti.

- **(1) Manuale installazione** — installazione del programma per Windows, linux o multiplatforma

- **(2) Manuale utente** — intercaccia utente di myAssistente
 - **(3) Manuale configurazione** — file `config.json` configurazioni avanzate
 - **(4) Manuale memoria** — file `memory.json` configurazioni avanzate
 - **(5) Manuale lingue** — file `lang[].json` customizzazioni avanzate
 - **(6) Manuale AIML** — file `aiml/[LANG]/[filename].aiml`
 - **(7) Manuale AIML avanzato** — file `aiml/IT/testConiglio.aiml`
-

Toolkit **myAssistente5** – Assistente desktop personale

[Sito dell'autore](#) · © 2026 [Emanuele Cassani](#)